ECLIPSE

Connected lamp built into a concrete plant pot — brief for the design of the electronic board

Breloque · August 2026

1. The project

ECLIPSE is a small decorative lamp built into the base of a concrete plant pot. It lights up using a tunable-white LED strip and is controlled from a browser over the Wi-Fi of an ESP32-C3: brightness, colour temperature, scheduling, and a sunrise/sunset effect. A physical button also allows the brightness to be adjusted directly, without going through Wi-Fi.

This is a personal project, in a small production run. I built the concept on a breadboard, then designed a board myself without being an electronics engineer. It was manufactured and assembled in 5 units, and **it burned out on first power-up**. I had the causes analysed afterwards: see section 4.

What I am looking for: someone to redo the design from scratch — schematic, layout, manufacturing files. You are entirely free regarding components, architecture and values.

> **A note about this document**
>
> I am not an electronics engineer. This brief describes **what I want to achieve** and **what I observed**, nothing more. The items in section 4 are either measurements taken from my own design files, or datasheet quotations for which I give the source. **I make no technical recommendations**: no values, no architecture, no calculations. That is your job — and my board failed precisely because those decisions were made by someone who was not qualified to make them.

> **The one thing that changes everything: the board is potted in resin**
>
> Once validated, the board is encapsulated in sealing resin. After that, **nothing can be repaired or reprogrammed**. Everything must therefore be testable and programmable before potting, and thermal behaviour must be assessed with no convection.

2. The original breadboard build

Before designing anything, I validated the concept on a breadboard. It works as follows.

A **12 V** supply feeds the circuit and serves two purposes. It powers the **LED strip** directly — a tunable-white (CCT) COB strip rated **12 W per metre at 12 V** — and it also feeds a **step-down regulator** that produces the **3.3 V** required by the **ESP32-C3**.

The strip has two channels, warm white and cool white, which is what allows the colour temperature to be varied. Each channel is switched by a **MOSFET** driven by the ESP32, which modulates the power of each channel. Adjusting both together gives control over brightness and tint.

A **button** is connected to the ESP32 to adjust brightness by hand. **This point matters to me:** the button must remain functional independently of Wi-Fi. If the network is unavailable, if no phone is present, or if the web interface is unresponsive, it must still be possible to switch the lamp on and adjust its brightness directly on the object itself.

That is all. The build comes down to five elements: the 12 V input, the regulator down to 3.3 V, the ESP32-C3, the two MOSFETs for the two strip channels, and the button.

3. What the board must do

Function	Requirement
Microcontroller	ESP32-C3, module with integrated antenna (e.g. `ESP32-C3-MINI-1` or equivalent). 2.4 GHz Wi-Fi in access-point mode.
LED output	2 independent dimmable channels (warm white + cool white), driven by the ESP32.
LED strip	12 V CCT COB, rated **12 W/m**, **1 m maximum**. That is 12 W, so 1 A at 12 V. I could not determine whether this 12 W/m figure applies to both channels combined or per channel: please verify against the strip's datasheet — I can supply the exact part reference.
Button	Manual brightness adjustment, **working without Wi-Fi**. Connected by two wires. On my board nothing was provided in hardware; everything relied on firmware.
Power	12 V DC input from an external mains adapter. The strip runs on 12 V; the ESP32 needs 3.3 V (Espressif tolerance: 3.0–3.6 V).
Protection	Reversed polarity at connection must not be able to destroy the board.
Programming	3.3 V serial access, usable after assembly — see section 5.
Connections	Three soldered-wire connections, no bulky connectors: 12 V input, LED strip (3 wires), button (2 wires). Please provide strain relief for the wires.

ESP32-C3 strapping pins

The three *strapping* pins are **floating by default** and the module **contains no pull resistors**. I left them unconnected, which is one of the faults on my board.

Pin	What the Espressif documentation states
`GPIO2`	Must be 1 at reset: no valid boot mode exists with GPIO2 at 0.
`GPIO8`	Must be held high to enter download mode reliably. The combination GPIO8 at 0 with GPIO9 at 0 is invalid.
`GPIO9`	1 = normal boot, 0 = download mode. Has an internal pull-up.

Sources: ESP32-C3 datasheet (strapping pin tables); ESP32-C3-MINI-1 datasheet (module schematic); esptool documentation (boot mode selection).

Size — as small and as thin as possible

> The board sits in the base of a concrete pot and is then potted in resin. **Every millimetre saved, in both area and thickness, genuinely matters to me** — it is a criterion I will look at in proposals, not merely a limit not to exceed.
>
> **58 × 24 mm is an absolute maximum, not a target.** If you can go smaller, that is very welcome. The same applies to thickness: I originally aimed for 0.8 mm, and my board ended up at 1.6 mm for mechanical strength. If thinner is workable, I would prefer it. Layer count is up to you: if going to 4 layers reduces the area, that is a good trade-off for me.

The module antenna must face a board edge with the keep-out area Espressif requires; Espressif also calls for a continuous ground plane under and around the RF section. *Source: ESP32 Hardware Design Guidelines.*

4. What went wrong on my board

Analysis carried out afterwards. Each item is either a **measurement taken from my design files** or a **datasheet quotation**. I propose no corrections — those decisions are yours. The board used an `MP2307` regulator, an `ESP32-C3-MINI-1-H4` module and two `AO3400A` MOSFETs.

1. **The regulator's EN pin was tied directly to +12 V. This is what destroyed the board.** The MP2307 datasheet gives, in its absolute maximum ratings: "*All Other Pins ... –0.3V to +6V*". This pin received the full input rail voltage. Immediate heating, then smoke.

2. **GPIO2 left unconnected**, although Espressif states that no valid boot mode exists with this pin at 0, and that it is floating by default with no pull resistor inside the module.
3. **GPIO8 left unconnected**, although it must be held high to enter download mode, and the combination GPIO8 at 0 with GPIO9 at 0 is described as invalid — while the flashing procedure specifically pulls GPIO9 low.
4. **The regulator's SS pin was left floating**, which the datasheet describes as follows: "*To disable the soft-start feature, leave SS unconnected*". Soft-start was therefore disabled. Also worth noting, two documented figures: the MP2307 over-voltage protection trips 20 % above the nominal output, and the ESP32-C3 is rated 3.6 V absolute maximum.
5. **The compensation resistor was set to 10 kΩ with no calculation**, although the datasheet provides a design procedure for this network which was never applied.
6. **The input capacitor sat 9.62 mm away from the regulator's input pin**, connected by a 0.4 mm trace, with no high-frequency decoupling nearby — while the datasheet requires minimising the switching current loop and placing decoupling as close as possible to the input pin. Also worth noting: the LED strip is fed from the same 12 V rail and its current is switched.
7. **The feedback divider resistors sat 14.6 mm and 17.8 mm from the relevant pin**, with 20.4 mm of trace on that signal, although the datasheet requires placing them "*as close to the chip as possible*".
8. **The bootstrap capacitor was set to 10 nF**, exactly the floor value stated ("*0.01μF or greater*"), with no margin above that minimum.
9. **The ESP32 module footprint allows neither inspection nor rework.** Generated by an automatic library conversion, its pads stop 0.30 mm inside the module outline on all three sides: no solder fillet is visible, and no pin can be reached without desoldering the whole module. This is what makes items 2 and 3 uncorrectable on my existing boards.
10. **The MP2307 is marked NRFND** — not recommended for new designs — by its manufacturer, which lists the MP2393 as its replacement. Its datasheet dates from 2008.
11. **Additional points.** No auto-reset circuit was provided: flashing required shorting one pad to ground by hand while triggering a reset on another. The input diode carries the entire current drawn by the board and the strip, which deserves review given that everything is potted in resin. The button had neither pull resistor nor hardware debouncing on the board.

Sources: MP2307 datasheet rev. 1.9 (Absolute Maximum Ratings, pin descriptions, Application Information, PCB Layout); Monolithic Power Systems product page; ESP32-C3 and ESP32-C3-MINI-1 datasheets; esptool documentation. Distances are measured from my own design files.

> **What I ask for in terms of method**
>
> No technical solution is imposed, but a few points matter to me: check every pin of every active component against the **absolute maximum ratings** table in its datasheet — this is the step that was skipped and that destroyed my board; explicitly handle the microcontroller's **strapping pins**; apply the **layout section** of the datasheet of whichever power component you choose; size the regulation networks **by calculation** using the manufacturer's procedure; use **manufacturer-recommended footprints** with inspectable pads; avoid end-of-life parts; and make sure everything is **testable and programmable before potting**.

5. Programming

The ESP32-C3 ships blank. The firmware, which I supply, is loaded **after assembly, over the serial link, and necessarily before the resin is poured**. If this access path is faulty, the board is scrap. The Espressif documentation states that programming is done over UART0 at **3.3 V**, and that to enter download mode the module must be reset **while GPIO9 is held low, with GPIO8 held high**.

What I would like: no more manual handling to flash a board. On my board, one pad had to be shorted to ground by hand while triggering a reset on another — awkward and unreliable. The solution is yours to choose; I simply want to be able to plug in the interface and start flashing. Please provide clearly marked access, usable after assembly; I had six pads on the back at 2.54 mm pitch (`TXD0, RXD0, EN, GPIO9, GND, 3V3`), which worked fine.

Order of operations before potting, to be achievable on every board: short-circuit check with power off, then first power-up with no load, measuring the logic rail and checking temperatures, then firmware programming, then Wi-Fi and web interface verification, then testing both LED channels separately and together at full load, then testing the button, then a sustained full-load thermal check — **and only then, the resin**.

6. Deliverables and manufacturing

- Schematic, routed PCB, manufacturing files (Gerber, BOM, CPL) **and editable source files** — the design belongs to me.
- **A table listing, for every active component, each pin with its absolute maximum rating from the datasheet alongside the voltage actually applied.** This is the check that was missing and that I want to be able to have reviewed.
- A note explaining your sizing choices for the power supply, with the supporting calculations.
- Clean DRC and ERC reports, with a comment on any remaining warnings, and the test procedure matching the sequence above with expected values.

Manufacturing: automated SMT assembly (JLCPCB or equivalent), **a 2-board prototype run first** before any batch. For every part, verify that the actual package matches the footprint: on an earlier prototype, an 0805 capacitor placed on a 1206 footprint blocked assembly.

Silkscreen: BRELOQUE logo (file supplied) and the ECLIPSE name; legible marking of the connections with explicit 12 V polarity; programming points labelled; and a **pin-1 marker on every integrated circuit** — it was missing on my board, and the assembler nearly fitted the regulator the wrong way round. Solder mask colour is open; a matte black would suit me. ENIG or lead-free HASL finish.

No CE marking on this version: it will be handled later, once actual conformity (EMC, RED for Wi-Fi) has been assessed. Please do not apply the CE mark at this stage.